MATERIAL DE APOIO · MÓDULOS 2 E 3

sf e terra



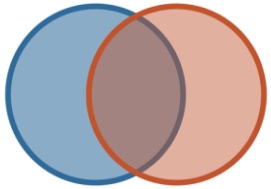
Cada pergunta tem uma construção vizinha

PREDICADO · PERGUNTA		OPERAÇÃO · CONSTRÓI
	`st_intersects(x, y)` algum ponto em comum? →	 `st_intersection(x, y)` a parte comum
	`st_disjoint(x, y)` nenhum ponto em comum? →	 `st_difference(x, y)` x menos a parte em y
	`st_is_within_distance(x, y, dist)` a até dist de y? →	 `st_buffer(y, dist)` a faixa de largura dist
	`st_nearest_feature(x, y)` qual y é o mais próximo? →	 `st_nearest_points(x, y)` a linha até o mais próximo
	`st_within(x, y)` x inteiro dentro de y? →	 `st_union(x)` tudo fundido em uma

■ x ■ y ··· entrada ■ geometria devolvida

st_intersects() pergunta; st_intersection() constrói

Predicado



```
sf::st_intersects(A, B)
```

Retorna: TRUE ou FALSE, ou os índices de B que tocam cada A

A e B saem intactos: nada é recortado

Operação

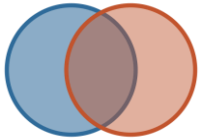


```
sf::st_intersection(A, B)
```

Retorna: Uma geometria nova: só o pedaço que A e B têm em comum

Os atributos de A e de B vão juntos para o resultado

O que cada predicado testa



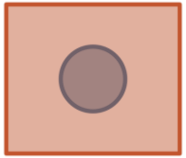
`st_intersects`

algum ponto em comum: o mais usado



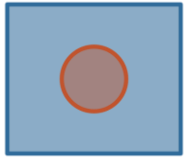
`st_disjoint`

nenhum ponto em comum: o contrário



`st_within(x, y)`

x inteiro dentro de y



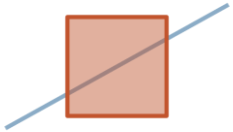
`st_contains(x, y)`

y inteiro dentro de x: o inverso



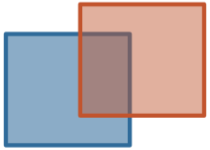
`st_touches`

só as bordas se encostam



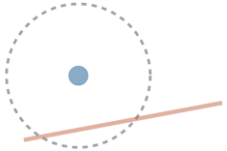
`st_crosses`

a linha atravessa o polígono



`st_overlaps`

sobreposição parcial, mesmo tipo de geometria

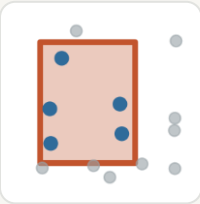


`st_is_within_distance`

a até dist um do outro

■ x ■ y Todos os desenhos foram gerados com o sf, e o predicado deu TRUE em cada um.

Três usos de um predicado



Filtrar

fica só quem responde TRUE

```
# portos dentro da faixa da rodovia
sf::st_filter(portos, faixa,
              .predicate = sf::st_within)
```



Contar

quantos y tocam cada x

```
# quantos portos em cada município
municipios$n_portos <- lengths(
  sf::st_intersects(municipios, portos))
```



Juntar atributos

x recebe as colunas de y

```
# cada porto recebe o seu município
sf::st_join(portos, municipios,
            join = sf::st_within)
```

As operações devolvem geometria nova

Entre duas geometrias



`st_intersection`



`st_difference`



`st_sym_difference`



`st_union(x, y)`

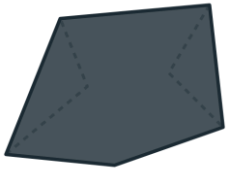
Sobre uma geometria



`st_buffer`



`st_centroid`



`st_convex_hull`



`st_simplify`

Tracejado: a entrada. Preenchido: o que a função devolve.

EXEMPLO COM OS DADOS DO CURSO

Portos perto das rodovias da Serra do Mar (PR)

```r
rod <- sf::st_read("dados/brutos/rodovias_serra_pr.gpkg") |>
  sf::st_transform(31982)          # metros, não graus
portos <- sf::st_read("dados/brutos/portos_osm.gpkg") |>
  sf::st_transform(31982)

# predicado: quais portos estão a até 5 km das rodovias?
perto <- sf::st_is_within_distance(portos, rod, dist = 5000)
portos[lengths(perto) > 0, ]

# operação: a própria faixa de 5 km, como polígono
faixa <- sf::st_buffer(sf::st_union(rod), 5000)
sf::st_area(faixa)
```

6

de 12 portos ficam a até 5 km das rodovias

7.469

km² na faixa de 5 km em volta das rodovias

Projete antes de medir. Em graus (4326), dist e área não fazem sentido.

sf para vetor, terra para raster

	sf
O dado	pontos, linhas e polígonos com atributos
O objeto	`sf (um data.frame)`
Ler	`st_read()`
Reprojetar	`st_transform()`
Recortar	`st_crop(), st_intersection()`
Resumir	`st_area(), st_length()`
Salvar	`st_write()`

sf para vetor, terra para raster

	sf	terra
O dado	pontos, linhas e polígonos com atributos	grade de células: elevação, imagem, uso do solo
O objeto	`sf (um data.frame)`	`SpatRaster`
Ler	`st_read()`	`rast()`
Reprojetar	`st_transform()`	`project()`
Recortar	`st_crop(), st_intersection()`	`crop() + mask()`
Resumir	`st_area(), st_length()`	`global(), zonal()`
Salvar	`st_write()`	`writeRaster()`

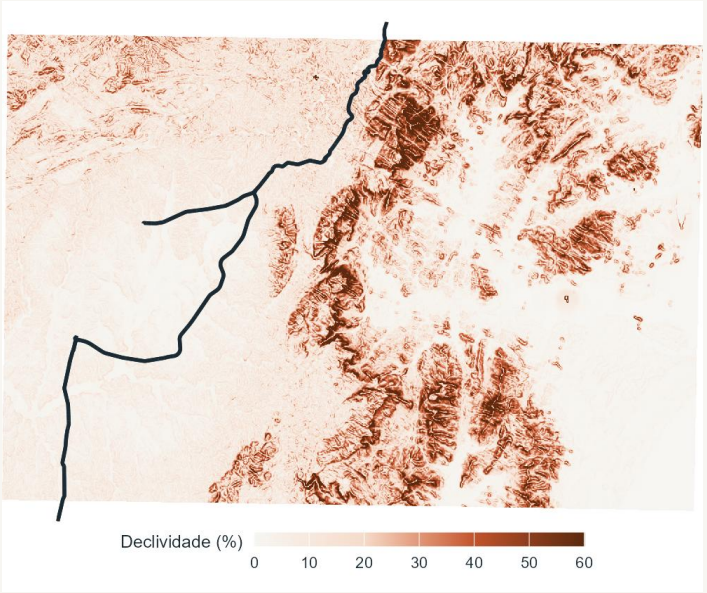
Da elevação à declividade ao longo da BR-116

```
# 1. ler o DEM e projetar em metros
dem <- terra::rast("dados/brutos/dem_serra.tif") |>
  terra::project("EPSG:31982")

# 2. declividade em %
decl <- tan(terra::terrain(dem, v = "slope",
                           unit = "radians")) * 100

# 3. a rodovia, no mesmo CRS
br116 <- sf::st_read(
  "dados/brutos/br116_serra.gpkg") |>
  sf::st_transform(31982)

# 4. valores do raster sob a linha
v <- terra::extract(decl, terra::vect(br116))
median(v[[2]], na.rm = TRUE)    # 4,7 %
```



Declividade na Serra do Mar (PR) e a BR-116. DEM do capítulo de declividade (elevatr).

As quatro pontes entre sf e terra

sf → SpatVector

`terra::vect(x)`

para usar uma geometria do sf em funções do terra

SpatVector → sf

`sf::st_as_sf(v)`

para voltar ao dplyr, ao ggplot2 e ao mapview

raster → tabela

`terra::extract(r, vect(x))`

os valores do raster sob cada ponto, linha ou polígono

vetor → raster

`terra::rasterize(vect(x), r)`

a geometria vira grade, na resolução do raster r

Antes de cruzar: o mesmo CRS nos dois lados. Confira com sf::st_crs(x) e terra::crs(r).

mapview para conferir, mapgl para apresentar

mapview

Leaflet: camadas desenhadas uma a uma no navegador

- Uma linha mostra qualquer sf ou SpatRaster
- Popup com todos os atributos, sem configurar nada
- Fica lento com dezenas de milhares de feições (o leafgl ajuda)

```
mapview::mapview(aop, zcol = "P001")
```

mapgl

MapLibre GL / Mapbox GL: desenho via WebGL

- Cada camada é declarada: cor, raio, opacidade, tooltip
- Aguenta bases grandes, estilos, extrusão 3D e transições
- Mais código, porém mais controle sobre o resultado

```
mapgl::maplibre(bounds = portos) |>
  mapgl::add_circle_layer(
    id = "portos", source = portos,
    tooltip = "nome")
```